

Problem Set 7

B. Magic Potion

PROBLEMS SUBMIT CODE MY SUBMISSIONS STATUS STANDINGS CUSTOM INVOCATION

My Submissions

#	When	Who	Problem	Lang	Verdict	Time	Memory
383766630	Jul/22/2026 10:55 ^{UTC+10}	tianzheyu	I - Magic Potion	C++23 (GCC 14- 64, msys2)	Accepted	46 ms	30100 KB

<https://codeforces.com/submissions/tianzheyu#>

Process

Implemented a maximum flow network approach, running Dinic's algorithm to find the maximum bipartite matching between heroes and monsters alongside a shared potion pool limit.

Challenges and Overcoming

1. When coding my flow network edge capacities, I ran into a bug where my program was outputting a maximum flow much larger than the correct answer. I found it when I ran the sample test case and saw that a single hero was able to kill multiple monsters well beyond their allowed limit. Tracing back the logic, I realized I had set the edge capacity from the potion node to the hero nodes to `network.INF` instead of 1, which inadvertently allowed a single hero to consume the entire potion pool. Next time to prevent this bug, I will explicitly map out the physical constraints of the problem next to my graph drawing and double-check every capacity assignment against these rules before typing the `add_edge` functions.
2. Later, when constructing the network topology, I ran into another bug where my program was failing to find valid augmenting paths. I found it when I reviewed my edge definitions and realized I had connected the source node `S` backwards, routing edges from the heroes to `S` instead of `S` to the heroes. I also accidentally routed the potion flow into the `hero_out` nodes, which had no outgoing connections to the monsters, creating a dead end. Next time to prevent this bug, I will mentally trace a complete path of the flow from the source all the way to the sink to verify the correct directionality of every edge before writing the code.